Windows Objects in Kernel Vulnerability Exploitation

By **Matthew "j00ru" Jurczyk**

Windows kernel vulnerabilities are continuously becoming more and more popular among security experts, in the recent years. This is probably caused by the fact that code running in the mysterious, *ring-0* mode has its own set of rules, as well as potential bugs. Moreover, the possible benefits of exploiting a kernel vulnerability are tremendously different from these, found in user-mode software. Such differences are a simple consequence of the operating system design itself – both processor modes are meant to be used by code responsible for various tasks, such as:

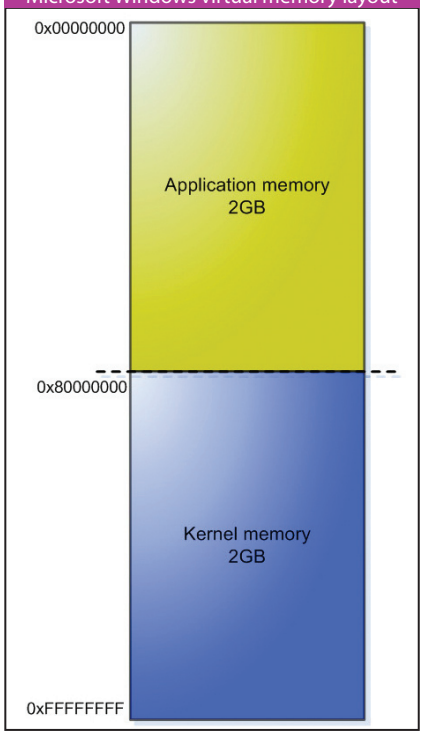
- Security management
- Providing a stable execution environment for user applications
- Physical device management
- Running user-specific programs, such as word processor, internet browser, games etc.

As can be seen, the first three points require considerably higher system privileges, than the latter one. Associating different code modules with different privileges is called *privilege separation*, and is a vital part of *Protected Mode* – the operational mode introduced in the Intel x86 processors in the early 90's. This paper aims to cover some of the possible ways of gathering sensitive data from the Windows kernel, and then using it to elevate the current application privileges, consequently leading to system security compromise.

Protected-Mode Basics

Before thinking of how the system privileges could be escalated by a potential attacker, one should firstly focus on some basic information about the *Protected Mode* design.

Diagram 1.
Microsoft Windows virtual memory layout



What has been mentioned in the previous section, various system tasks require multiple privilege levels to work on. Thus, in order to provide fair system security, less critical modules should be assigned lower privileges, while the more critical ones should run with full control over the system. To achieve this, Intel introduced four privilege levels (so-called *rings*) - with *ring-0* being the most, and *ring-3* less privileged mode. In practice, most of the modern operating systems only take advantage of *ring-0* and *ring-3*, leaving the remaining two levels unused. Hence, two types of code can be distinguished – *kernel* code (which is not limited to the kernel image, only), having almost complete control over the machine (virtualization mechanisms are beyond the scope of this paper) and *user* code, most commonly executed by ordinary applications, used by the user himself.

One of the most revolutionary features brought by *Protected Mode* was memory protection. As opposed to *Real Mode*, it is now possible for the system to maintain the total, available physical memory in a convenient manner. The address space size increased from 20 to 32 bits (1 megabyte to 4 gigabytes). Furthermore, as the virtual addressing was distract from physical addressing, the OS was

eventually able to separate the memory areas utilized by numerous, active processes.

However, all the features found in new CPU series would remain useless, if the operating systems didn't support these features in the *software way*. Hence, the authors of the operating systems had to design a reasonable security model, based on the *Protected Mode* improvements. The general idea, used in Windows until today, is shown in Image 1. As the image presents, the entire virtual addressing is split into two major parts – *user- and kernel-memory*.

The lower part of the address space is purposed to be accessed by user's applications. As mentioned before, all the programs working on Windows are taking advantage of virtual memory separation – in other words, every single process can operate on his own 2 gigabytes of memory, without sharing it with any other program – this part of memory is *process-specific*. A natural consequence is that user memory is swappable – can be swapped out and saved on the hard disk, when the system is running out of physical memory. Due to the fact that these memory regions are used by non-privileged modules, they can be accessed from within all rings.

The higher part, on the other hand, *belongs* to modules running under *ring-0*. It can be accessed by the system code, only – ordinary applications are unable to execute, modify, or even read its contents. These regions are system-wide, thus don't change on thread switch, but remain the same regardless of the current process. Gaining the ability to execute *ring-0* code makes it possible to subvert the system security, i.e. by installing a stealth *rootkit*, or performing other malicious operations. The entire security design is based on preventing an usual user from altering the existing kernel code or executing his own.

Even though user applications are meant to execute with the *ring-3* rights, a great number of operations cannot be achieved without employing some system management functions, placed in the kernel areas. As noted, it is impossible to directly call privileged code, due to the memory access restrictions. However, a few transition mechanisms have been developed, allowing *ring-3* to *ring-0* transitioning, such as:

- System calls (SYSENTER/SYSEXIT instructions)
- Interrupts (INT instruction)
- Call Gates (CALL FAR instruction)

All of the above methods let the application call a pre-defined kernel function with a certain number of parameters. In case of syscalls, the system must previously initialize an adequate *Model-specific register* (MSR), interrupts require a valid *Interrupt Descriptor Table* to be present, while Call Gates are based on the *Global/Local Descriptor Table*. As can be seen, all of the methods take advantage of structures managed by the system itself. The user is unable to mess with either GDT or IDT – these structures reside inside kernel memory – or MSR, as the *Write MSR* (WMSR) instruction is reserved for *ring-0* mode.

As shown, probably the only possible way of elevating the security privileges would require finding and exploiting a vulnerability present in a kernel function, that is able to be called by a (potentially hostile) user application.

The Real Value Of Kernel Addresses

Having some elementary knowledge of how Protected Mode works, one could ask about how the kernel addresses could prove useful for an user-mode application, since the process wouldn't be able to access data under that address, after all. On the other hand, numerous vulnerabilities are being found in device drivers, and a majority of them can be classified as *write-what-where conditions*. This particular kind of bug makes it possible to, literally, use the vulnerable driver to write a specified value (**what**) to a chosen location (**where**). Such a situation might be a consequence of many possible scenarios, like lack of input/output pointer sanity checks, *pool-based* buffer overflows, and so on. In order to gain *ring-0* code execution, one must first choose the appropriate *what* and *where* operands, so that the write operation leads to the desired result.

For the last couple of years, various critical memory locations (playing the where role) have been researched and described in detail. This includes places, such as **nt!KiDebugRoutine**[1], **nt!HalDispatchTable**[2] (exported), **nt!MmUserProbeAddress**[3] (exported), or even the kernel code instructions, themselves! Some of the above methods turned out to be stable and solid, while other remained in the hypothetical state only. One way or another, all of them pose a very interesting subject for further investigation.

Windows Objects

In order to provide consistent access to various resources made available by the operating system, Windows implements a specific object model. As Windows *Internals 5* states[4], the object manager (a part of the Windows kernel responsible for object management) was designed to meet the following goals:

- Provide a common, uniform mechanism for using system resources,
- Isolate object protection to one location in the operating system so that C2 security compliance can be achieved,
- Provide a mechanism to charge processes for their use of objects so that limits can be placed on the usage of system resources,
- Establish an object-naming scheme that can readily incorporate existing objects, such as the devices, files, and directions of the file system, or other independent collections of objects,

Listing 1. Definition of the OBJECT_HEADER structure on Windows 7 RC x86

```
nt!_OBJECT_HEADER
   +0x000 PointerCount       : Int4B
   +0x004 HandleCount        : Int4B
   +0x004 NextToFree         : Ptr32 Void
   +0x008 Lock               : _EX_PUSH_LOCK
   +0x00c TypeIndex          : UChar
   +0x00d TraceFlags         : UChar
   +0x00e InfoMask           : UChar
   +0x00f Flags              : UChar
   +0x010 ObjectCreateInfo   : Ptr32 _OBJECT_CREATE_INFORMATION
   +0x010 QuotaBlockCharged  : Ptr32 Void
   +0x014 SecurityDescriptor : Ptr32 Void
   +0x018 Body               : _QUAD
```

- Support the requirements of various operating system environments,
- Establish uniform rules for object retention,
- Provide the ability to isolate objects for a specific session to allow for both *local* and *global* objects in the namespace.

In this paper, we are mostly interested in the *executive* objects, commonly (yet indirectly) utilized by user-mode applications through the Windows API. Some examples of such objects are: files, directories, threads, processes or events. These resources can be tampered with, using functions like *CreateFile*, *WriteFile*, *OpenProcess*, *SetEvent* etc. Each of the above object types represents a certain system resource.

Internally, Windows objects are implemented as basic structures, containing *type-specific* information. Since these structures are stored inside kernel memory, and thus no application has direct access to its contents, all the desired operations are performed by the kernel, on behalf of the user's program. However, *ring-3* code doesn't operate on raw kernel addresses – instead, special values called Handles are provided by the Object Manager. These *handles* are actually indexes into the *Process Handle Table*, which in turn contains pointers to the associated structures. In other words, handles are used as the user-mode representatives of system resources, and are translated to real pointers in the kernel mode.

The internal object structure is composed of two integral parts – the object header, common for all existing types of objects, and the latter part – *object-specific* data. The object header includes information such as its name, security descriptor, quota charges and other, standard characteristics. More precisely, it is described by a structure named OBJECT_HEADER, presented in *Listing 1*.

After 24 bytes of the above properties, a next structure follows, depending on the object type. Most of the *executive* object structures are defined in the Microsoft Debugging Symbols[5] for the *ntoskrnl.exe* image. Some exemplary, widely used structure names are: *KPROCESS* (process), *KTHREAD* (thread) or *KSEMAPHORE* (semaphore). More detailed definitions of a few objects are presented later in this paper.

Retrieving object-related information from within user-mode

As mentioned before, every single internal object structure is safely stored in the high memory regions, protected from unauthorized write access. Despite that, as it turns out, Windows operating system provides multiple services (system calls), designed to supply a variety of information regarding the current system state. A list of the most important information-querying functions follows:

- **NtQuerySystemInformation**[6] – returns system-wide information, such as kernel configuration (e.g. *memory pools*), hardware information (e.g. processor characteristics), global system settings (e.g. current time), and much more,
- **NtQueryInformationProcess**[7] – returns information about a certain process, based on internal process structures like *KPROCESS*,
- **NtQueryInformationThread**[8] – same as above, involving the thread object,
- **NtQueryJobObject, NtQueryInformationToken, NtQueryInformationPort** and other – return *type-specific* information about a specific Windows object.

A majority of the *NtQueryInformation~* functions have their counterparts – *NtSetInformation~* - responsible for changing the specified information instead of querying for it. However, among all the available information classes (defined in *ddk\winddk.h* and *ddk\ntapi.h*, can also be found in the *Windows NT 2000 Native API Reference*[9] book), some of them are marked read-only, while other can be changed, as well. Because of the fact that most of the information related to objects is obtained and set using the above routines, they are extensively used by multiple external libraries, such as *kernel32*.dll, which utilize these system calls to implement documented Windows API functions.

The **NtQuerySystemInformation** function along with **SystemHandleInformation** parameter can be used to obtain data regarding all open handles present in the system. On a valid call, the function returns a 32-bit unsigned integer – *NumberOfHandles* – and the appropriate number of *SYSTEM_HANDLE_TABLE_ENTRY_INFO* structures, each describing a single handle. The definitions of both structures are shown in *Listing 2*.

After successfully reading structures of all the existing system handles, one can easily extract the address of a certain object. The problem is even simpler, when the handle is created in the context of the local process – in this case, both *UniqueProcessId* and *HandleValue* fields are known straight away, which is enough to find the right descriptor structure. *Listing 3* shows an

Listing 2. Definitions of the structures return by the NtQuerySystemInformation system call

```
typedef struct _SYSTEM_HANDLE_TABLE_ENTRY_INFO {
  USHORT UniqueProcessId;
  USHORT CreatorBackTraceIndex;
  UCHAR ObjectTypeIndex;
  UCHAR HandleAttributes;
  USHORT HandleValue;
  PVOID Object;
  ULONG GrantedAccess;
} SYSTEM_HANDLE_TABLE_ENTRY_INFO, *PSYSTEM_HANDLE_TABLE_ENTRY_INFO;

typedef struct _SYSTEM_HANDLE_INFORMATION {
ULONG NumberOfHandles;
SYSTEM_HANDLE_TABLE_ENTRY_INFO Handles[ 1 ];
} SYSTEM_HANDLE_INFORMATION, *PSYSTEM_HANDLE_INFORMATION;
```

Where:

UniqueProcessId
The Process ID of the owner of the handle.

CreatorBackTraceIndex
Debugging purpose field, usually zero.

ObjectTypeIndex
The object type identifier of the handle in consideration.

HandleAttributes
Contains internal flags, specifying the handle properties (such as *PROTECTED_FROM_CLOSE*).

HandleValue
The exact handle value, that the owner process is operating on.

Object
The kernel-mode address of the object referred by the handle.

GrantedAccess
Access granted at the time of creating the handle.

Listing 3. An exemplary function, retrieving the virtual address of a specified object

```
LPVOID GetHandleAddress(ULONG dwProcessId, USHORT hObject)
{
  NTSTATUS NtStatus;
  SYSTEM_HANDLE_INFORMATION SystemHandle;
  BYTE* HandleInformation;
  DWORD BytesReturned = 0;
  ULONG i;

NtQuerySystemInformation(SystemHandleInformation,
&SystemHandle,sizeof (SYSTEM_HANDLE_INFORMATION), &BytesReturned);

  HandleInformation = new BYTE[BytesReturned];
  if(!HandleInformation)
    return NULL;

if(!NT_SUCCESS(NtQuerySystemInformation(SystemHandleInformation,
HandleInformation,BytesReturned,&BytesReturned)))
  {
    delete HandleInformation;
    return NULL;
  }

  PSYSTEM_HANDLE_INFORMATION HandleInfo = (typeof(HandleInfo))
HandleInformation;
  PSYSTEM_HANDLE_TABLE_ENTRY_INFO CurrentHandle = &HandleInfo-
>Handles[0];

  for( i=0;i<HandleInfo->NumberOfHandles;CurrentHandle++,i++ )
  {
    if(CurrentHandle->UniqueProcessId == dwProcessId &&
       CurrentHandle->HandleValue     == (USHORT)hObject)
    {
      LPVOID ReturnAddr = CurrentHandle->Object;
      delete HandleInformation;
      return ReturnAddr;
    }
  }

  delete HandleInformation;
  return NULL;
}
```

exemplary function, extracting the object structure address based on the two values detailed above.

In practice, one is able to obtain the address of any object, regardless of its type – the only requirement here is that the process in consideration created a handle to the resource, and we know its numeric value. Being able to find any given object, let's proceed to the next step.

Some particular Windows objects in practice

In the *Introduction* section of this paper, I mentioned that before exploiting a *write-what-where* vulnerability, one must find a place that – when overwritten – would lead us straight to a privilege elevation. In other words, appropriate fields, such as function pointers, must be found in the object structures to compromise the machine. Additionally, one must be able to get the kernel to use the modified pointer – this, however, doesn't pose a serious problem.

Out of nearly 30 *executive* objects, three objects that illustrate the idea best are described here. These objects are **Timer (KTIMER)**, **Thread (KTHREAD)**, **Process (KPROCESS)**. It is possible to find a few more structures, containing very sensitive fields – keep in mind that overwriting a function pointer is not a necessity. Modifying other, less "ordinal" values could be also a good solution in many cases.

Timer Object

The first target on our way to achieve privileged code execution is a Waitable Timer Object. As the MSDN documentation states[10]:

A waitable timer object is a synchronization object whose state is set to signaled when the specified due time arrives. There are two types of waitable timers that can be created: manual-reset and synchronization. A timer of either type can also be a periodic timer.

This mechanism has been present in Microsoft Windows since the very beginning of NT series, and hasn't changed too much during the past few years. Some of the most important API functions utilized by legitimate user-mode applications, include:

- **CreateWaitableTimer** and **CreateWaitableTimerEx** for creating the object,
- **SetWaitableTimer** for setting the object configuration, such as the interval time, timer period, optional callback routines, and so on. Internally, this function is responsible for the actual modification of the kernel object contents,
- **CancelWaitableTimer** to deactivate the mechanism and **CloseHandle** to entirely give up using the particular object.

Keeping the above names in mind, it's also important to know what system calls are employed while using documented API functions – these are **NtCreateTimer** and **NtOpenTimer** for requesting access to an existing timer or creating one from scratch, **NtSetTimer** for changing the object settings, **NtCancelTimer** for deactivating a chosen timer.

Because of the fact that every Windows object does have its own *type-specific* structure, so have the timers. To be more exact, all the internal timer-management functions operate on a common structure definition – see *Listing 4*.

Listing 4. The KTIMER structure definition

```
nt!_KTIMER
 +0x000 Header                    : _DISPATCHER_HEADER
 +0x010 DueTime                   : _ULARGE_INTEGER
 +0x018 TimerListEntry            : _LIST_ENTRY
 +0x020 Dpc                       : Ptr32 _KDPC
 +0x024 Period                    : Uint4B
```

Listing 5. The KDPC structure definition

```
nt!_KDPC
   +0x000 Type                          : UChar
   +0x001 Importance                    : UChar
   +0x002 Number                        : Uint2B
   +0x004 DpcListEntry                  : _LIST_ENTRY
+0x00c DeferredRoutine                  : Ptr32 void
+0x010 DeferredContext                  : Ptr32 Void
+0x014 SystemArgument1                  : Ptr32 Void
+0x018 SystemArgument2                  : Ptr32 Void
+0x01c DpcData                          : Ptr32 Void
```

Listing 6. The exploitation target inside the KTHREAD structure

```
ntdll!_KTHREAD
   +0x000 Header                        : _DISPATCHER_HEADER
   +0x010 CycleTime                     : Uint8B
   +0x018 HighCycleTime                 : Uint4B
   +0x020 QuantumTarget                 : Uint8B
   (...)
   +0x18a OtherPlatformFill             : UChar
   +0x18c Win32Thread                   : Ptr32 Void
   +0x190 StackBase                     : Ptr32 Void
   +0x194 SuspendApc                    : _KAPC
   +0x194 SuspendApcFill0               : [1] UChar
   +0x195 ResourceIndex                 : UChar
   +0x194 SuspendApcFill1               : [3] UChar
   +0x197 QuantumReset                  : UChar
   +0x194 SuspendApcFill2               : [4] UChar
   +0x198 KernelTime                    : Uint4B
   (...)
```

At a first glance, one might not see any value that could be worth being beneficially overwritten. The important fact, however, is that the DPC acronym stands for *Deferred Procedure Call*, a popular *kernel-mode* Windows mechanism allowing high-priority task to schedule a procedure to be executed later in time, with lower priority. And so, the *KDPC* structure definition does contain fields that are indeed worth being changed – see *Listing 5*. The pointer to the deferred function is placed inside the DeferredRoutine field, found at offset 0x0C (12d).

As shown, having control over the internal *KTIMER* structure would let a potential attacker execute a *ring-0* payload, by forwarding the *Dpc* pointer to the user-mode part of memory, where a new, malicious KDPC structure could be easily crafted.

Thread Object

The next structure that, after being altered, brings certain benefits, is the structure responsible for storing information about a single thread present in the system. As a relatively complex mechanism, a number of various information regarding every thread must be kept in memory, such as information about *user-* and *kernel-* mode stacks, *Thread Environment* Block pointer, multiple flags, execution priority, processor affinity, and much more. The most interesting part of the *KTHREAD* structure, however, is one specific field called *SuspendApc*, a pointer to the *KAPC* structure. Let's find out what this name stands for!

The APC (*Asynchronous Procedure Call*) mechanism[11] allows system modules to queue a procedure to be called in the context of a chosen thread, either in *ring-3* or *ring-0* mode. Such a procedure is described by the *KAPC* structure which, in turn, is put onto a special *thread-specific* queue. When an appropriate moment comes (i.e. when the thread enters an alerted state, for example by using the *SleepEx*[12] API function), the procedures are called respectively, and their corresponding structures are erased from the queue – most often, until the queue is entirely empty.

The question is – what does it have to do with the *SuspendApc* field in our structure?

Since Windows NT times, a mechanism called *thread suspension* has been supported by the Windows API. This basically means that most threads, belonging to ordinary applications can remain in two, opposite states: active and inactive. In case of the first one, the thread's execution is normally scheduled, based on its affinity, priority, general system state and numerous other factors. In the latter case, however, the thread is considered *frozen* – the operating system doesn't schedule its execution, its current stack contents/processor context doesn't change etc.

Suspending and resuming threads can be achieved by using the **SuspendThread**[13] and **ResumeThread**[14] API functions or, more internally, **NtSuspendThread** along with **NtResumeThread**. The most interesting part of this mechanism is the actual way, of how the execution of an active thread is being suspended after calling an adequate function.

On thread creation, the *KeInitThread* function initializes the *SuspendApc* field with some pre-defined values, which don't change until the thread termination. After that, when an external process decides to suspend our thread, the already-initialized *KAPC* structure is put on the APC queue belonging to the thread in consideration. The *NormalRoutine* function – *KiSuspendThread* in this case – is then immediately called in the context of the target thread. When the procedure returns, the thread is already suspended. The interesting part of how the mechanisms works is the fact that the user is able to:

1. Retrieve the virtual address of a specified thread's *KTHREAD* structure, and hence the *SuspendApc* field too,
2. Indirectly (through system calls) call the function pointer defined in *KAPC*

If additionally, the user knew a way of overwriting certain kernel memory areas (i.e. using a vulnerable device driver), the *KTHREAD* structure could be successfully utilized in the vulnerability exploitation process.

One thing that should be noted is that using the thread suspension mechanism is being advised against even by Microsoft itself, as it might cause serious stability problem in the context of the application with suspended threads.

The technique covered in this chapter was first described by *skape & Skywing* in the "a catalog of windows local kernel-mode backdoors" article[15].

Process Object

Another object that could be taken into consideration while exploiting a *write-what-where* vulnerability could be the process itself. Just like threads, processes – special *containers* responsible for providing common execution environment (such as memory context) to multiple threads – must also be described by a variety of different parameters. These include kernel / user execution times, thread list, flags, affinity and others. For a complete listing of the *KPROCESS* structure definition, see *Listing 8*.

A variety of fields that could be taken advantage of, can be observed. In this particular case, however, I would like to focus on *LdtDescriptor*.

The Intel x86 architecture supports two types of Descriptor Tables: the Global and Local ones. While GDT is a *per-processor* structure, there can be multiple LDTs available on the system. More precisely, Windows allows at most one LDT to be associated with a single process. Due to the fact that the decision whether to use the local table or not is up to the application itself – it is an optional feature. As a consequence, every process is started without LDT – it can be created and maintained by the system on demand.

The descriptor table management functions are scattered between the Win32 (*kernel32.dll*) and undocumented, native (*ntdll.dll*) API. When one wants to employ the LDT mechanism, he can choose between calling **NtSetInformationProcess** and **NtSetLdtEntries** (both from the Native API set). On the other hand, querying for information about existing descriptors is accomplished by using either **GetThreadSelectorEntry**[16] (Win32 API) or **NtQueryInformationProcess** (Native API).

Because of the volatile nature of LDTs (which have to be changed every time the process context is switched), the system does have to safely store the descriptor, so that it can be copied into GDT when desired, but wouldn't be accessible by the application's code, at the same time – the *KPROCESS* structure seems to be a perfect place for this purpose, and so it is!

Listing 7. The *SuspendApc* field initialization

```
PAGELK:0071221D     push     ebx
PAGELK:0071221E     push     ebx
PAGELK:0071221F     push     offset _
KiSuspendThread@12
PAGELK:00712224     push     offset _xHalPrepareForBugcheck@4
PAGELK:00712229     push     offset _KiSuspendNop@20
PAGELK:0071222E     push     ebx
PAGELK:0071222F     push     esi
PAGELK:00712230     lea      eax, [esi+194h]
PAGELK:00712236     push     eax
PAGELK:00712237     call     _KeInitializeApc@32
```

Or, translated into pseudo-code:

```
KeInitializeApc(KTHREAD->SuspendApc, KTHREAD, 0, KiSuspendNop,
xHalPrepareForBugcheck, KiSuspendThread, 0, 0);
```

Listing 8. The *KPROCESS* structure definitionn

```
+0x000 Header                           : _DISPATCHER_HEADER
+0x010 ProfileListHead                  : _LIST_ENTRY
+0x018 DirectoryTableBase               : Uint4B
+0x01c LdtDescriptor                    : _KGDTENTRY
+0x024 Int21Descriptor                  : _KIDTENTRY
+0x02c ThreadListHead                   : _LIST_ENTRY
+0x034 ProcessLock                      : Uint4B
+0x038 Affinity                         : _KAFFINITY_EX
+0x044 ReadyListHead                    : _LIST_ENTRY
+0x04c SwapListEntry                    : _SINGLE_LIST_ENTRY
+0x050 ActiveProcessors                 : _KAFFINITY_EX
+0x05c AutoAlignment                    : Pos 0, 1 Bit
+0x05c DisableBoost                     : Pos 1, 1 Bit
+0x05c DisableQuantum                   : Pos 2, 1 Bit
+0x05c ActiveGroupsMask                 : Pos 3, 1 Bit
+0x05c ReservedFlags                    : Pos 4, 28 Bits
+0x05c ProcessFlags                     : Int4B
+0x060 BasePriority                     : Char
+0x061 QuantumReset                     : Char
+0x062 Visited                          : UChar
+0x063 Unused3                          : UChar
+0x064 ThreadSeed                       : [1] Uint4B
+0x068 IdealNode                        : [1] Uint2B
+0x06a IdealGlobalNode                  : Uint2B
+0x06c Flags                            : _KEXECUTE_OPTIONS
+0x06d Unused1                          : UChar
+0x06e IopmOffset                       : Uint2B
+0x070 Unused4                          : Uint4B
+0x074 StackCount                       : _KSTACK_COUNT
+0x078 ProcessListEntry                 : _LIST_ENTRY
+0x080 CycleTime                        : Uint8B
+0x088 KernelTime                       : Uint4B
+0x08c UserTime                         : Uint4B
+0x090 VdmTrapcHandler                  : Ptr32 Void
```

As presented in the "GDT and LDT in Windows kernel vulnerability exploitation"[17], having at least partial control over a segment descriptor may tremendously affect the system security. A potential attacker could try to transform an existing *LDT-type* descriptor into a *ring-0 Call Gate*, or redirect the existing LDT into user-space memory, where further steps would be taken to elevate the execution privileges.

Compatibility

When it comes to kernel-mode exploitation, what counts most is the compatibility across as great number of system versions, as possible. Let's reflect about whether the techniques presented above, or any other attacks based on overwriting the contents of Windows objects, could be used to develop a stable exploit. The actual exploitation process consists of three major parts: retrieving a certain object's address, preparing data used to overwrite the object, and sending a proper signal to the vulnerable device driver (or modifying the kernel memory by other means).

The presented method of enumerating all handles present in the system – *NtQuerySystemInformation* with the *SystemHandleInformation* parameter is valid for every Windows NT version known by the author, and can be treated as a reliable source of handle-related information. However, obtaining the base address of the object is just the first phase of calculating the virtual address of a particular field. The second part requires a correct offset to be added to the base, which could result in compatibility-related problems. As Microsoft is removing, adding, and changing existing features in both *user-* and *kernel-mode*, the offsets in internal (especially non-documented) structures tend to change very frequently. One possible solution to this problem would be to hardcode offsets from all the *exploit-supported* Windows versions and check the version before performing any WRITE operation in the kernel. Another option would require the attacker to use a *relatively stable* structure, such as **KTIMER**, which hasn't changed since decades.

As for the destination data preparation, the real compatibility depends on the object type of our choice. Although, in most cases, the desired result is having a function pointer modified, and then getting the kernel to call it – in such a situation, no compatibility issues may occur (the function pointer of the attacker's payload doesn't have to be *formed* in any way). The very last part of the actual attack – sending the "launch signal" to the kernel module in consideration - doesn't pose any problem in the compatibility context.

Taking the above facts into consideration, the only potential, significant issue would regard *object-specific* offsets that could possibly vary from one system version to another – as shown, multiple countermeasures can be taken in order to eliminate this problem. Therefore, methods presented in this paper can be considered relatively stable, in comparison to other, existing techniques.

Conclusion

In this paper, the author wanted to present a general idea of what parts of the Windows kernel could be successfully treated as an attack vector when combined with *extra abilities* (such as overwriting small parts of kernel memory), most often a consequence of a security vulnerability in one of the device drivers.

Out of all the existing possibilities, only three possible attack vectors has been chosen and described in detail. For sure, a great number of other, interesting (more or less) targets exist – finding and testing them out is left as an exercise for the reader. Furthermore, one could probably find other ways of overwriting the structures covered in this document, e.g. by tampering with other fields. The overall idea, however, remains the same.

Happy vulnerability hunting! •

REFERENCES

1. Ruben Santamarta: *Exploiting Common Flaws In Drivers*, http://www.reversemode.com/index.php?option=com_content&task=view&id=38&Itemid=1
2. Kostya Kortchinsky: *Real World Kernel Pool Exploitation*, http://sebug.net/paper/syscanhk/KernelPool.pdf
3. SoBeIt: *How to exploit Windows kernel memory pool*, http://packetstormsecurity.nl/Xcon2005/Xcon2005_SoBeIt.pdf
4. Mark Russinovich, David A. Solomon, Alex Ionescu: *Windows Internals 5*
5. Microsoft, Debugging Tools and Symbols
6. Sven B. Schreiber, Tomasz Nowak: NtQuerySystemInformation, http://undocumented.ntinternals.net/UserMode/Undocumented%20Functions/System%20Information/NtQuerySystemInformation.html
7. Sven B. Schreiber, Tomasz Nowak: NtQueryInformationProcess, http://undocumented.ntinternals.net/UserMode/Undocumented%20Functions/NT%20Objects/Process/NtQueryInformationProcess.html
8. Tomasz Nowak: NtQueryInformationThread, http://undocumented.ntinternals.net/UserMode/Undocumented%20Functions/NT%20Objects/Thread/NtQueryInformationThread.html
9. Gary Nebbett: Windows NT/2000 Native API Reference
10. MSDN: *Waitable Timer Objects*, http://msdn.microsoft.com/en-us/library/ms687012(VS.85).aspx
11. MSDN: *Asynchronous Procedure Calls*, http://msdn.microsoft.com/en-us/library/ms681951(VS.85).aspx
12. MSDN: *SleepEx Function*, http://msdn.microsoft.com/en-us/library/ms686307(VS.85).aspx
13. MSDN: *SuspendThread Function*, http://msdn.microsoft.com/en-us/library/ms686345(VS.85).aspx
14. MSDN: *ResumeThread Function*, http://msdn.microsoft.com/en-us/library/ms685086(VS.85).aspx
15. skape & Skywing: *A Catalog of Windows Local Kernel-mode Backdoor Techniques*, http://www.uninformed.org/?v=8&a=2&t=sumry
16. MSDN: *GetThreadSelectorEntry Function*, http://msdn.microsoft.com/en-us/library/ms679363(VS.85).aspx
17. Matthew „j00ru" Jurczyk, Gynvael Coldwind: *GDT and LDT in Windows kernel vulnerability exploitation*, vexillium.org/dl.php?call_gate_exploitation.pdf